

Voice Control Machine Code Tracking

1. Function Description

Note: When tracking an object, it needs to be moved slowly within the camera's field of view. This case only has tracking, no grasping function!

After all programs are running, the robotic arm will move to the tracking pose. At this time, hold the AprilTag and place it in the camera field of view. Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Track and clip machine code" to the voice module. Slowly move the AprilTag, and the robotic arm will track the AprilTag.



2. Startup and Operation

```
# Start camera and inverse kinematics
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition and announcement
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control
ros2 run dofbot_voice_ctrl grasp_VC
# Start machine code tracking
ros2 run dofbot_voice_ctrl apriltag_follow_2D
```

3. Core Code Analysis

Machine Code Tracking Node apriltag_follow_2D.py

Source code path:

```
/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/AprilTag/apriltag_follow_2D.py
```

This section mainly describes how to subscribe to and process voice recognition result topics.

```

# Create subscriber for voice recognition result topic
self.sub_voice =
self.create_subscription(Int8,"voice_result",self.getVoiceResultCallback,1)
# Callback function, processes received voice recognition result topic data. If
the received voice recognition result topic data is 105, it means the command is
"Track and grasp AprilTag". Change self.start_flag to True to indicate that the
AprilTag position information can be published.
# 回调函数，处理接收到的语音识别结果话题数据。如果接收到的语音识别结果话题数据是105，表示命令
为"追踪夹取机器码"，改变self.start_flag为True表示可以发布机器码的位置信息
def getVoiceResultCallback(self,msg):
    def getVoiceResultCallback(self,msg):
        if msg.data == 105:
            self.start_flag = True
        elif msg.data == 76:
            self.start_flag = False
            self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)

```

Robotic arm tracking

```

def XY_track(self, center_x, center_y):
    # The current detected object's center position
    # 当前检测到的物体中心位置
    self.px = center_x
    self.py = center_y

    # X-axis Servo Control
    # X轴舵机控制
    if not (self.target_servox >= 180 and center_x <= 320 and self.a == 1 or
            self.target_servox <= 0 and center_x >= 320 and self.a == 1):
        # Normal tracking mode
        # 正常跟踪模式
        if self.a == 0:
            # Set the current object's x-coordinate as the PID system output
            # 将当前物体的x坐标设为PID系统输出
            self.xservo_pid.SystemOutput = center_x

            # Handle out-of-range situation
            # 处理超出范围的情况
            if self.x_out_range == True:
                # Force the servo to the boundary position
                # 强制舵机到达边界位置
                if self.target_servox < 0:
                    self.target_servox = 0
                    self.xservo_pid.SetStepSignal(630)
                if self.target_servox > 0:
                    self.target_servox = 180
                    self.xservo_pid.SetStepSignal(10)
                self.x_out_range = False
            else:
                # Set the step signal to the image center x (320)
                # 将步进设为图像中心x坐标（320）
                self.xservo_pid.SetStepSignal(320)
                self.x_out_range = False

        # Set the inertia time parameters for the PID controller
        # 设置PID控制器的惯性时间参数

```

```

self.xservo_pid.SetInertiaTime(0.01, 0.1)

# Calculate the target value for the servo
# 计算舵机的目标值
target_valuex = int(1500 + self.xservo_pid.SystemOutput)

# Convert the target value to a servo angle
# 将目标值转换为舵机角度
self.target_servox = int((target_valuex - 500) / 10) - 10

# Check if the calculated angle exceeds the valid range
# 检查计算出的角度是否超出有效范围
if self.target_servox > 180:
    self.x_out_range = True
if self.target_servox < 0:
    self.x_out_range = True

# Y-axis Servo Control
# Y轴舵机控制
# Similar boundary check for the y-axis
# 对y轴进行类似的边界检查
if not (self.target_servoy >= 180 and center_y <= 240 and self.b == 1 or
        self.target_servoy <= 0 and center_y >= 240 and self.b == 1):
    if self.b == 0:
        # Set the current object's y-coordinate as the PID system output
        # 将当前物体的y坐标设为PID系统输出
        self.yservo_pid.SystemOutput = center_y

        # Handle out-of-range situation
        # 处理超出范围的情况
        if self.y_out_range == True:
            self.yservo_pid.SetStepSignal(450)
            self.y_out_range = False
        else:
            # set the step signal to the image center y (240)
            # 将步进设为图像中心y坐标 (240)
            self.yservo_pid.SetStepSignal(240)

    # Set the inertia time parameters for the PID controller
    # 设置PID控制器的惯性时间参数
    self.yservo_pid.SetInertiaTime(0.01, 0.1)

    # Calculate the target value for the servo
    # 计算舵机的目标值
    target_valuey = int(1500 + self.yservo_pid.SystemOutput)

    # Enforce a minimum value to prevent the arm from going too low
    # 强制执行最小值，防止机械臂降得太低
    if target_valuey <= 1000:
        target_valuey = 1000
        self.y_out_range = True

    # Convert the target value to a servo angle
    # 将目标值转换为舵机角度
    self.target_servoy = int((target_valuey - 500) / 10) - 55

    # Clamp the angle to the valid range [0, 180]

```

```
# 将角度限制在有效范围[0, 180]内
if self.target_servoy > 180:
    self.target_servoy = 180
if self.target_servoy < 0:
    self.target_servoy = 0
# Calculate the joint angle
# 计算关节角度
joint2 = 120 + self.target_servoy
joint3 = self.target_servoy / 4.5
joint4 = self.target_servoy / 3

joints_0 = [float(self.target_servox / 1), float(joint2), float(joint3),
float(joint4), 90.0, 30.0]

# Send the joint commands to the robotic arm
# 向机械臂发送关节控制命令
self.Arm.Arm_serial_servo_write6_array(joints_0, 2000)
```